Pandas Functions

In [ ]:
```
import pandas as pd
import numpy as np

data = {"class":["A","B","C","A","B","C"],
        "name":["Ram Lal","Kebin","Sonchita","Romi","Dusa","Satukshi"],
        "city":["Bihar","Nepal","Korea","Germany","Haryana","UP"],
        "days":["Monday","Tuesday","Wednusday","Thursday","Friday","Saturday"],
        "sales":["23345","2574752","54387","643756","3587","866397"]}
df2 = pd.DataFrame(data)            # student/sales table from Day 11
```

```
Iris.csv created from sklearn (150 x 6).
```

13. `pd.read_csv()` — Loading external data

Description: Reads a CSV file into a DataFrame — the standard entry point for any analysis.

In [ ]:
```
df = pd.read_csv("Iris.csv")
df.head()
```

Out[ ]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	setosa
1	2	4.9	3.0	1.4	0.2	setosa
2	3	4.7	3.2	1.3	0.2	setosa
3	4	4.6	3.1	1.5	0.2	setosa
4	5	5.0	3.6	1.4	0.2	setosa

In [ ]:
```
df = pd.read_csv("Iris.csv", index_col="Id")
df.head()
```

Out[ ]:

Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa

In [ ]:
```
df = pd.read_csv("Iris.csv", usecols=["SepalLengthCm","Species"])
df.head()
```

Out[ ]:

	SepalLengthCm	Species
0	5.1	setosa
1	4.9	setosa
2	4.7	setosa
3	4.6	setosa
4	5.0	setosa

Reload a full copy of Iris for the remaining demos (Impl 3 above left only two columns loaded).

In [5]:
```
df = pd.read_csv("Iris.csv")
```

14. `pd.DataFrame()` — Building a DataFrame from scratch

Description: Turns a dictionary (keys = column names, values = lists) into a table. This is how you made `df2`.

In [ ]:
```
df2 = pd.DataFrame(data)
df2
```

Out[ ]:

	class	name	city	days	sales
0	A	Ram Lal	Bihar	Monday	23345
1	B	Kebin	Nepal	Tuesday	2574752
2	C	Sonchita	Korea	Wednusday	54387
3	A	Romi	Germany	Thursday	643756
4	B	Dusa	Haryana	Friday	3587
5	C	Satukshi	UP	Saturday	866397

In [ ]:
```
rows = [{"class":"A","name":"Ram Lal"}, {"class":"B","name":"Kebin"}]
df3 = pd.DataFrame(rows)
df3
```

Out[ ]:

	class	name
0	A	Ram Lal
1	B	Kebin

In [ ]:
```
df4 = pd.DataFrame(np.array([[1,2],[3,4]]), columns=["x","y"])
df4
```

Out[ ]:

	x	y
0	1	2
1	3	4

15. Displaying a DataFrame (`df`, `head()`, `tail()`)

Description: Writing `df` alone shows the whole frame (with `...` in the middle). `head()` peeks at the top rows, `tail()` at the bottom — quick sanity checks after loading.

In [9]:
```
# Impl 1: full display + default head/tail (first/last 5 rows)
print(df)
print()
print(df.head())
print()
print(df.tail())              # your cell [10]
```

```
      Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species
0      1            5.1           3.5            1.4           0.2      setosa
1      2            4.9           3.0            1.4           0.2      setosa
2      3            4.7           3.2            1.3           0.2      setosa
3      4            4.6           3.1            1.5           0.2      setosa
4      5            5.0           3.6            1.4           0.2      setosa
..   ...            ...           ...            ...           ...         ...
145  146            6.7           3.0            5.2           2.3   virginica
146  147            6.3           2.5            5.0           1.9   virginica
147  148            6.5           3.0            5.2           2.0   virginica
148  149            6.2           3.4            5.4           2.3   virginica
149  150            5.9           3.0            5.1           1.8   virginica

[150 rows x 6 columns]

   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm Species
0   1            5.1           3.5            1.4           0.2  setosa
1   2            4.9           3.0            1.4           0.2  setosa
2   3            4.7           3.2            1.3           0.2  setosa
3   4            4.6           3.1            1.5           0.2  setosa
4   5            5.0           3.6            1.4           0.2  setosa

      Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm    Species
145  146            6.7           3.0            5.2           2.3  virginica
146  147            6.3           2.5            5.0           1.9  virginica
147  148            6.5           3.0            5.2           2.0  virginica
148  149            6.2           3.4            5.4           2.3  virginica
149  150            5.9           3.0            5.1           1.8  virginica
```

In [10]:
```
# Impl 2: custom number of rows
print(df.head(10))            # first 10
```

```
print()
print(df.tail(3))              # last 3
```

```
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm Species
0   1            5.1           3.5            1.4           0.2  setosa
1   2            4.9           3.0            1.4           0.2  setosa
2   3            4.7           3.2            1.3           0.2  setosa
3   4            4.6           3.1            1.5           0.2  setosa
4   5            5.0           3.6            1.4           0.2  setosa
5   6            5.4           3.9            1.7           0.4  setosa
6   7            4.6           3.4            1.4           0.3  setosa
7   8            5.0           3.4            1.5           0.2  setosa
8   9            4.4           2.9            1.4           0.2  setosa
9  10            4.9           3.1            1.5           0.1  setosa

      Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm    Species
147  148            6.5           3.0            5.2           2.0  virginica
148  149            6.2           3.4            5.4           2.3  virginica
149  150            5.9           3.0            5.1           1.8  virginica
```

In [11]:
```
# Impl 3: programmatic access to the rows
first_rows = df.iloc[:5]      # slice-based head
last_row   = df.iloc[-1:]     # just the final record
print(first_rows)
print()
print(last_row)
```

```
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm Species
0   1            5.1           3.5            1.4           0.2  setosa
1   2            4.9           3.0            1.4           0.2  setosa
2   3            4.7           3.2            1.3           0.2  setosa
3   4            4.6           3.1            1.5           0.2  setosa
4   5            5.0           3.6            1.4           0.2  setosa

      Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm    Species
149  150            5.9           3.0            5.1           1.8  virginica
```

16. `df.info()` — Metadata summary

Description: Prints the class, index range, every column's non-null count and dtype, plus memory usage — your screenshot showing 150 entries, 6 columns, dtypes `float64` / `int64` / `str`. Best first command after loading data (finds missing values instantly).

In [12]:
```
# Impl 1: standard call
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   Species        150 non-null    str
dtypes: float64(4), int64(1), str(1)
memory usage: 7.2 KB
```

In [13]:
```python
# Impl 2: limit displayed rows (useful for huge datasets)
df.info(max_cols=3)
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Columns: 6 entries, Id to Species
dtypes: float64(4), int64(1), str(1)
memory usage: 7.2 KB
```

In [14]:
```python
# Impl 3: rebuild the same info manually
for c in df.columns:
    print(c, "| non-null:", df[c].notna().sum(), "| dtype:", df[c].dtype)
```

```
Id | non-null: 150 | dtype: int64
SepalLengthCm | non-null: 150 | dtype: float64
SepalWidthCm | non-null: 150 | dtype: float64
PetalLengthCm | non-null: 150 | dtype: float64
PetalWidthCm | non-null: 150 | dtype: float64
Species | non-null: 150 | dtype: str
```

17. `df.describe()` — Statistical summary

Description: Auto-computes count, mean, std, min, 25%, 50% (median), 75%, max for all numeric columns — exactly the table in your cell [13].

In [15]:
```python
# Impl 1: default (numeric columns only)
df.describe()
```

Out[15]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.057333	3.758000	1.199333
std	43.445368	0.828066	0.435866	1.765298	0.762238
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

In [16]:
```
# Impl 2: include text columns too (gives count/unique/top/freq)
df.describe(include="all")
```

Out[16]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Specie
count	150.000000	150.000000	150.000000	150.000000	150.000000	15
unique	NaN	NaN	NaN	NaN	NaN	
top	NaN	NaN	NaN	NaN	NaN	setos
freq	NaN	NaN	NaN	NaN	NaN	5
mean	75.500000	5.843333	3.057333	3.758000	1.199333	NaI
std	43.445368	0.828066	0.435866	1.765298	0.762238	NaI
min	1.000000	4.300000	2.000000	1.000000	0.100000	NaI
25%	38.250000	5.100000	2.800000	1.600000	0.300000	NaI
50%	75.500000	5.800000	3.000000	4.350000	1.300000	NaI
75%	112.750000	6.400000	3.300000	5.100000	1.800000	NaI
max	150.000000	7.900000	4.400000	6.900000	2.500000	NaI

In [17]:
```
# Impl 3: pick specific statistics
print(df.describe(percentiles=[.10, .90]))          # add 10th & 90th percentile
print()
print(df[["SepalLengthCm","PetalWidthCm"]].describe())   # subset of columns
```

```
                Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
count   150.000000     150.000000    150.000000     150.000000    150.000000
mean     75.500000       5.843333      3.057333       3.758000      1.199333
std      43.445368       0.828066      0.435866       1.765298      0.762238
min       1.000000       4.300000      2.000000       1.000000      0.100000
10%      15.900000       4.800000      2.500000       1.400000      0.200000
90%     135.100000       6.900000      3.610000       5.800000      2.200000
max     150.000000       7.900000      4.400000       6.900000      2.500000

       SepalLengthCm  PetalWidthCm
count     150.000000    150.000000
mean        5.843333      1.199333
std         0.828066      0.762238
min         4.300000      0.100000
25%         5.100000      0.300000
50%         5.800000      1.300000
75%         6.400000      1.800000
max         7.900000      2.500000
```

18. `df.shape` — Dimensions of the table

Description: Attribute (no parentheses!) returning `(rows, columns)`. Yours was `(150, 6)`.

In [18]:
```
# Impl 1: direct attribute
df.shape                 # (150, 6)
```

Out[18]:
```
(150, 6)
```

In [19]:
```
# Impl 2: separate the two numbers
n_rows, n_cols = df.shape
print(n_rows, n_cols)
```

```
150 6
```

In [ ]:
```
print(len(df))
print(len(df.columns))
```

```
150
6
```

19. `df.columns` / `.to_list()` — Working with column names

Description: `.columns` holds the `Index` of column names; `.to_list()` converts it to a plain Python list so you can count, loop, or filter over it.

In [ ]:
```
len(df.columns.to_list())
```

Out[ ]:
```
6
```

In [ ]:
```
df.rename(columns={"SepalLengthCm":"SL","Species":"Class"}, inplace=True)
print(df.columns.to_list())
```

```
['Id', 'SL', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm', 'Class']
```

In [ ]:
```
petal_cols = [c for c in df.columns if "Petal" in c]
df[petal_cols]
```

Out[ ]:

	PetalLengthCm	PetalWidthCm
0	1.4	0.2
1	1.4	0.2
2	1.3	0.2
3	1.5	0.2
4	1.4	0.2
...	...	...
145	5.2	2.3
146	5.0	1.9
147	5.2	2.0
148	5.4	2.3
149	5.1	1.8

150 rows × 2 columns

Reload a fresh copy of Iris (Impl 2 above renamed columns in place, which changes the rest of Part B).

In [24]:
```
df = pd.read_csv("Iris.csv")
```

20. `df.drop()` — Removing columns or rows

Description: Deletes specified columns (`columns=`) or rows (`index=`). By default it returns a new DataFrame (original untouched) unless you pass `inplace=True` . Your cell [20] dropped the `Id` column.

In [ ]:
```
df.drop(columns="Id").head()
print("df still has 'Id':", "Id" in df.columns)
```

```
df still has 'Id': True
```

In [ ]:
```
df.drop(columns=["Id","SepalWidthCm"], inplace=True)
df.head()
```

Out[ ]:

	SepalLengthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	1.4	0.2	setosa
1	4.9	1.4	0.2	setosa
2	4.7	1.3	0.2	setosa
3	4.6	1.5	0.2	setosa
4	5.0	1.4	0.2	setosa

Reload again — the example above mutated `df`.

In [27]:
```
df = pd.read_csv("Iris.csv")
```

In [ ]:
```
print(df.drop(index=[0,1,2]).head())
print()
df.drop("Id", axis=1)
```

```
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm Species
3   4            4.6           3.1            1.5           0.2  setosa
4   5            5.0           3.6            1.4           0.2  setosa
5   6            5.4           3.9            1.7           0.4  setosa
6   7            4.6           3.4            1.4           0.3  setosa
7   8            5.0           3.4            1.5           0.2  setosa
```

Out[ ]:

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

21. Column selection & filtering (`df[col]`, `df.loc`, `df.iloc`)

Description: Grabbing one column gives a `Series`; boolean filters like `df[df.Class=='A']` give matching rows. `loc` selects by label, `iloc` by integer position.

In [ ]:
```
print(df2["name"])
print()
print(df2[df2["class"] == "A"])
```

```
0     Ram Lal
1       Kebin
2    Sonchita
3        Romi
4        Dusa
5    Satukshi
Name: name, dtype: str

  class     name     city      days   sales
0     A  Ram Lal    Bihar    Monday   23345
3     A     Romi  Germany  Thursday  643756
```

In [ ]:
```
df.loc[0:5, ["SepalLengthCm","Species"]]
```

Out[ ]:

	SepalLengthCm	Species
0	5.1	setosa
1	4.9	setosa
2	4.7	setosa
3	4.6	setosa
4	5.0	setosa
5	5.4	setosa

In [ ]:
```
df.iloc[0:5, 0:3]
```

Out[ ]:

	Id	SepalLengthCm	SepalWidthCm
0	1	5.1	3.5
1	2	4.9	3.0
2	3	4.7	3.2
3	4	4.6	3.1
4	5	5.0	3.6